# CyberLabX V1 Roadmap

None

```
Epic 1
Project Foundation

Epic 2
Authentication

Epic 3
Dashboard

Epic 4
Experiment Engine

Epic 5
Lab Engine

Epic 6
Browser Terminal

Epic 7
Auto Grading

Epic 8
Reports

Epic 9
Faculty Dashboard

Epic 10
Admin Dashboard

Epic 11
Deployment
```

---

# EPIC 1 — Project Foundation

---

## Ticket 1.1

### Feature Name

Initialize Monorepo

### Description

Create the CyberLabX repository using a client/server architecture. Configure React + TypeScript + Vite for the frontend and Node.js + Express + TypeScript for the backend. Add ESLint, Prettier, Husky, Git hooks, and Docker Compose. Create a shared `.env.example` and initial README.

### Acceptance Criteria

- React app starts successfully.
- Express API starts successfully.
- Docker Compose runs supporting services.
- Linting and formatting pass.
- Repository structure matches the architecture document.

### Dependencies

None

### Priority

**Must-Have**

---

## Ticket 1.2

### Feature Name

Configure Tailwind Design System

### Description

Implement the CyberLabX design system in Tailwind using the approved colors, typography, spacing, border radius, shadows, and dark theme. Expose them as reusable design tokens.

### Acceptance Criteria

- Global styles applied.
- Theme matches design specification.
- Responsive layout enabled.
- Reusable utility classes documented.

### Dependencies

Ticket 1.1

### Priority

**Must-Have**

---

# Ticket 1.3

### Feature Name

Application Layout

### Description

Build the global layout including sidebar, top navigation, content container, breadcrumb support, and responsive behavior.

### Acceptance Criteria

- Layout works across all pages.
- Sidebar collapses/expands.
- Navigation highlights current route.
- Layout reusable for all authenticated pages.

### Dependencies

1.1

1.2

### Priority

Must-Have

---

# EPIC 2 — Authentication

---

## Ticket 2.1

### Feature Name

User Registration

### Description

Create a registration flow supporting Student and Faculty accounts. Validate input, hash passwords using bcrypt/Argon2, store users securely, and prevent duplicate email registrations.

### Acceptance Criteria

- Registration succeeds with valid data.
- Duplicate emails rejected.
- Passwords stored as hashes.
- Validation errors displayed clearly.

### Dependencies

1.1

Database setup

### Priority

Must-Have

---

## Ticket 2.2

### Feature Name

User Login

### Description

Implement email/password login using JWT stored in HttpOnly cookies. Include secure session handling and account lockout after repeated failures.

### Acceptance Criteria

- Valid users authenticate successfully.
- Invalid credentials rejected.
- JWT cookie created securely.
- Session persists across refresh.

### Dependencies

2.1

### Priority

Must-Have

---

# Ticket 2.3

### Feature Name

Role-Based Access Control

### Description

Implement middleware enforcing Student, Faculty, and Admin permissions for all protected routes and UI elements.

### Acceptance Criteria

- Unauthorized requests denied.
- Navigation updates based on role.
- Protected APIs require valid permissions.

### Dependencies

2.2

### Priority

Must-Have

---

## Ticket 2.4

### Feature Name

Forgot Password

### Description

Build a secure password reset flow using time-limited reset tokens delivered via email.

### Acceptance Criteria

- Reset email generated.
- Token expires automatically.
- Password updated after verification.

### Dependencies

2.2

### Priority

Should-Have

---

# EPIC 3 — Student Dashboard

---

## Ticket 3.1

### Feature Name

Student Dashboard

### Description

Build the main student dashboard displaying welcome information, experiment progress, completed labs, scores, and quick actions.

### Acceptance Criteria

- Dashboard loads after login.
- Experiment cards display correctly.
- Progress statistics visible.
- Responsive layout.

### Dependencies

Authentication

### Priority

Must-Have

---

## Ticket 3.2

### Feature Name

Experiment Catalog

### Description

Display available cybersecurity experiments with title, difficulty, estimated duration, completion status, and launch action.

### Acceptance Criteria

- Experiments loaded from API.
- Search and filtering available.
- Status indicators visible.

### Dependencies

3.1

### Priority

Must-Have

---

# EPIC 4 — Experiment Engine

---

## Ticket 4.1

### Feature Name

Experiment Content System

### Description

Create a content engine capable of rendering objectives, theory, procedures, images, and ordered sections dynamically from the database.

### Acceptance Criteria

- Experiment content renders correctly.
- Rich text supported.
- Easy to extend for future labs.

### Dependencies

Database

### Priority

Must-Have

---

## Ticket 4.2

### Feature Name

Nmap Experiment

### Description

Implement the complete Nmap experiment including objectives, theory, guided tasks, hints, expected commands, quiz, and report generation.

### Acceptance Criteria

- Student completes the full experiment.
- Tasks progress sequentially.
- Results stored correctly.

### Dependencies

Experiment Engine

### Priority

Must-Have

---

# EPIC 5 — Lab Engine

---

## Ticket 5.1

### Feature Name

Docker Lab Provisioning

### Description

Automatically create an isolated Docker container for each student session with the required networking and tools.

### Acceptance Criteria

- Container created successfully.
- Resources limited.
- Internal network configured.
- Container removed after completion.

### Dependencies

Backend

Docker

### Priority

Must-Have

---

## Ticket 5.2

### Feature Name

Lab Session Management

### Description

Manage creation, reconnection, expiration, timeout, and cleanup of student lab sessions.

### Acceptance Criteria

- One active lab per student.
- Session recovery supported.
- Automatic cleanup performed.

### Dependencies

5.1

### Priority

Must-Have

---

# EPIC 6 — Browser Terminal

---

# Ticket 6.1

## Feature Name

Web Terminal

## Description

Embed xterm.js and connect it to the backend using WebSockets for real-time command execution.

## Acceptance Criteria

- Commands execute correctly.
- Output streamed live.
- Terminal reconnects automatically.

## Dependencies

Lab Engine

## Priority

Must-Have

---

# Ticket 6.2

## Feature Name

Terminal Controls

## Description

Add copy, paste, clear, fullscreen, download session, and resize functionality.

## Acceptance Criteria

All controls function correctly.

## Dependencies

6.1

### Priority

Should-Have

---

# EPIC 7 — Auto Grading

---

## Ticket 7.1

### Feature Name

Command Validator

### Description

Evaluate submitted commands against experiment requirements, award marks, and provide contextual feedback.

### Acceptance Criteria

- Commands validated.
- Partial marks supported.
- Feedback generated.

### Dependencies

Experiment Engine

Terminal

### Priority

Must-Have

---

## Ticket 7.2

### Feature Name

Quiz Engine

### Description

Create reusable quiz functionality supporting multiple question types and automatic scoring.

### Acceptance Criteria

- Quiz loads dynamically.
- Scores calculated automatically.
- Results stored.

### Dependencies

Experiment Engine

### Priority

Must-Have

---

# EPIC 8 — Reports

---

## Ticket 8.1

### Feature Name

PDF Report Generator

### Description

Generate a downloadable PDF summarizing experiment details, executed commands, quiz results, grading, and timestamps.

### Acceptance Criteria

- PDF generated successfully.
- Correct formatting.

- Download available.

### Dependencies

Auto Grading

### Priority

Must-Have

---

## Ticket 8.2

### Feature Name

Progress Analytics

### Description

Display historical performance, completion trends, and average scores for each student.

### Acceptance Criteria

Charts render correctly.

### Dependencies

Reports

### Priority

Should-Have

---

# EPIC 9 — Faculty Dashboard

---

## Ticket 9.1

### Feature Name

Faculty Dashboard

### Description

Provide instructors with student progress, experiment completion statistics, recent activity, and grading overview.

### Acceptance Criteria

- Dashboard loads.
- Metrics accurate.
- Filters work.

### Dependencies

Authentication

Reports

### Priority

Must-Have

---

## Ticket 9.2

### Feature Name

Experiment Management

### Description

Allow faculty to create, edit, publish, archive, and assign experiments.

### Acceptance Criteria

CRUD operations complete.

### Dependencies

Experiment Engine

### Priority

Should-Have

---

# EPIC 10 — Admin

---

## Ticket 10.1

### Feature Name

User Management

### Description

Create administrative tools for managing students, faculty, and administrators, including activation and suspension.

### Acceptance Criteria

CRUD operations complete.

### Dependencies

Authentication

### Priority

Must-Have

---

## Ticket 10.2

### Feature Name

System Monitoring

### Description

Display server status, Docker health, active lab sessions, and resource utilization.

### Acceptance Criteria

Health metrics update correctly.

### Dependencies

Lab Engine

### Priority

Should-Have

---

# EPIC 11 — Deployment

---

## Ticket 11.1

### Feature Name

Production Deployment

### Description

Prepare the application for deployment using Docker Compose, Nginx, HTTPS, environment configuration, and production builds.

### Acceptance Criteria

- HTTPS enabled.
- Containers start successfully.
- Reverse proxy configured.
- Environment variables documented.

### Dependencies

All core features

### Priority

Must-Have

---

## Ticket 11.2

### Feature Name

Logging & Monitoring

### Description

Implement centralized logging, request tracing, error monitoring, and health checks for production.

### Acceptance Criteria

- Errors captured.
- Health endpoint available.
- Logs searchable.

### Dependencies

Deployment

### Priority

Should-Have

---

# Nice-to-Have Tickets (Post-MVP)

These are intentionally deferred until after the Nmap MVP is complete:

- Network packet visualization
- AI learning assistant
- Wireshark experiment
- SQL Injection lab
- XSS lab
- Password cracking lab
- Digital forensics lab
- Achievement badges

- Certificates
- Leaderboards
- LMS integration (Moodle, Google Classroom)
- Multi-language support
- Mobile companion app

---

# Recommended Sprint Plan

Instead of building feature-by-feature randomly, I'd organize the work into six sprints:

| Sprint | Goal |
|---|---|
| Sprint 1 | Project setup, authentication, design system |
| Sprint 2 | Student dashboard and experiment engine |
| Sprint 3 | Docker lab engine and browser terminal |
| Sprint 4 | Nmap experiment, command validation, quiz |
| Sprint 5 | Reports, faculty dashboard, admin tools |
| Sprint 6 | Testing, security hardening, deployment, documentation |

This order minimizes blockers. For example, there's little value in building PDF reports before grading exists, or a browser terminal before lab provisioning works.

One additional recommendation: after creating these tickets, define a **Definition of Done (DoD)** that applies to every ticket (e.g., code reviewed, tests passing, linting clean, documentation updated, no critical security issues). That keeps quality consistent as the project grows and makes the tickets directly usable with AI coding tools or human developers.